

TECHNICAL RESEARCH REPORT

JARVIS Builds Deep Dive

13 builds, researched fresh — what actually works,
what to steal, and the build plan



NAVIGATION

Table of contents

01	How this was researched	3	02	1. Executive summary	3
03	2. The 13 builds	4	04	Build 1 — Zubair (AI Workshop): JARVIS on Claude Fable 5.1 + Claude Code	4
05	Build 2 — SouvikChai: "from-scratch" Python ML voice assistant	5	06	Build 3 — manujademin: 100% offline JARVIS in Python	5
07	Build 4 — Software Gent: turn Hermes Agent into JARVIS (free)	6	08	Build 5 — Nate Herkman: no-code JARVIS (Lovable + ElevenLabs + n8n)	7
09	Build 6 — AIDDEVGENIUS: no-code JARVIS on Android mobile	8	10	Build 7 — @avoshuu: "Arcus," the local-first Chief Executive AI Officer	9
11	Build 8 — @chloeshy.ai: the business JARVIS	10	12	Build 9 — @alexcandothat: the DIY pipeline ("intelligence = context + relevancy")	11
13	Build 10 — @lukebuildsai: the 6am call (Azaris)	12	14	Build 11 — @blakeestephens: the driving call	12
15	Build 12 — @realrobertgutierrez: "Ghost X1" (abliterated model)	13	16	Build 13 — @jonathonmj: "Z.E.R.O"	14
17	3. The combined architecture: best of all 13	14	18	4. Substance-vs-marketing scorecard	16
19	5. What NOT to copy	17	20	6. Build plan — phased for a non-programmer with AI coding tools	17
21	Phase 1 — Weekend V1: "The Remembering Assistant" (~\$5–10/mo)	17	22	Phase 2 — Voice + money tools (~\$25–50/mo, only when Phase 1 is boring)	18
23	Phase 3 — The operator (one-time hardware ~\$100–200 + tokens)	18	24	V1 vs. later — stated plainly	19
25	Cost cheat sheet (verified ranges, Sept 2026)	19	26	7. Open questions to verify before building	20

Reading key: **VERIFIED** means independently supported; **PLAUSIBLE** means consistent but unconfirmed; **MARKETING** means unsupported or contradicted.

How this was researched

Each of the 13 videos was investigated from primary sources in September 2026: YouTube descriptions and chapter lists, GitHub repositories and READMEs (one repository's code was read file by file), Instagram captions, comment threads, and creators' follow-up posts pulled through the Instagram API, plus independent verification of every named component — models, MCP servers, TTS engines, agent frameworks, and pricing. Findings are labeled **VERIFIED** (independent evidence), **PLAUSIBLE** (consistent but unconfirmed), or **MARKETING** (no evidence, or contradicted by evidence). Inferred mechanics are marked as inferred. Nothing here is recycled from earlier summaries.

1. Executive summary

Thirteen builds, one convergence: every serious attempt lands on the same seven-layer architecture — a reasoning model, a memory system, a tools layer, a scheduled agent loop, a voice pipeline, multiple surfaces, and privacy tiers. The model is never the differentiator. **The differentiator is always memory plus tools: what the system remembers about you, and what it's allowed to touch.** That is the "intelligence = context + relevance" thesis, and the research confirms it.

The single most important recommendation: **build the memory pipeline on a real agent runtime first** — an always-on harness with persistent memory, a few tools, and a WhatsApp interface — before spending a dollar on voice realism, dashboards, or hardware. Everything else is decoration until the system remembers you across weeks and acts without being asked.

The seven ideas worth stealing:

1. **Alex's capture → transcribe → graph memory pipeline** (record everything, transcribe locally, keep it in a linked notes graph) — the practical implementation of "intelligence = context + relevance."
2. **A Hermes/OpenClaw-class agent runtime** (MIT, free): ~47 built-in tools, cron scheduling, SQLite session memory with full-text search, messaging gateways, model-agnostic — the only free foundation that is a genuine agent, not a chatbot.
3. **Chloe's MCP business-data loop** — RevenueCat, Meta Ads, and Playwright MCP servers turning business metrics into spoken operational briefs.
4. **Zubair's morning brief + cited answers** — Gmail + Calendar briefings and a "show me the exact note this came from" provenance habit.
5. **Nate's ElevenLabs + n8n voice-tool pattern** — the honest no-code realtime voice stack (voice agent calls tools via webhooks).
6. **Z.E.R.O's operator framing** — purpose, tools, memory, permissions, feedback loops. An LLM is a tool; JARVIS is an operator.
7. **manujademin's offline voice loop** (openWakeWord + faster-whisper + Ollama + Piper) — the privacy tier for sensitive stuff.

What to ignore: four of the thirteen are funnels with no verifiable system (Blake's course, Ghost X1's vaporware model, Z.E.R.O's consulting, Luke's scripted ad for a real but separate product). One celebrated repo is copied code with a toy classifier. "Fully local" and "uncensored" are marketing words until code or weights appear.

2. The 13 builds

Build 1 — Zubair (AI Workshop): JARVIS on Claude Fable 5.1 + Claude Code

<https://www.youtube.com/watch?v=YK-qJwmwjVc>

Verdict: The most capable architecture shown — and the least inspectable. Real components, gated internals, affiliate funnel around it.

What it claims: A JARVIS rebuilt on Claude Fable 5.1 using Claude Code: a 3D "knowledge galaxy" second brain generated from a folder of markdown notes, free two-way browser voice, webcam and screen vision, a self-verification harness, morning briefings from real Gmail and Calendar, Mac control by voice, mid-conversation "brain hot-swap" between Claude, Grok, and GPT, auto-generated invoices, and real outbound phone calls. Built by pasting 8 prompts into Claude Code; the finished build is sold through Skool.

How it actually works (reconstructed): The real engine is **Claude Code as the builder, not the assistant**. The prompts generate a local app that becomes the runtime: (1) microphone → Web Speech API recognition (free, Chrome) → text; (2) a backend sends text plus tools to an LLM endpoint — Anthropic's API with Fable 5.1 by default; (3) the "brain hot-swap" is just switching the API base URL and model string (e.g., to OpenRouter) — a configuration change, not a transplant; (4) notes are parsed into a local embedding index, queried semantically, and rendered as a three.js-style 3D galaxy; (5) webcam/screen frames go to the multimodal endpoint as vision input; (6) replies are spoken via free browser speech synthesis; (7) Gmail/Calendar via Google OAuth; (8) **phone calls are outsourced to Retell AI**, a cloud voice-agent plus telephony service — not his code; (9) the "self-verification harness" is a generated test script running canned checks.

Stack: LLM: Claude Fable 5.1 via Anthropic API (**VERIFIED** real release, Sept 1 2026: 1M-token context; pricing \$10/M input, \$50/M output, cache reads \$0.25/M). STT/TTS: free Web Speech API, both directions. Vision: webcam/screen frames → multimodal model. Memory: markdown notes → local embedding index (exact store undisclosed). Tools: Google OAuth (Gmail/Calendar), Retell AI for calls. Hardware: his Mac; PC/Mac compatible. **No public repo** — prompts gated behind Skool (free "lite" tier, finished install paid).

Evidence: Real: Fable 5.1 confirmed via independent press; Retell AI is a real priced service; free browser voice maps exactly to the Web Speech API; demos shown per chapters. Gloss: prompts are gated so "one prompt does the work of three" can't be verified; no independent reproductions found anywhere.

Cost & limits: API tokens for an always-on frontier model burn fast; Retell runs ~\$0.11–0.16/min realistic mid-configuration (\$2/mo phone number, \$10 free credits). Browser STT/TTS is mediocre — no reliable wake word or barge-in, Chrome-dependent. Gmail/Calendar OAuth is the biggest real friction. Mac control needs accessibility permissions and breaks across OS updates. Vision streaming is token-hungry.

Build 2 — SouvikChai: "from-scratch" Python ML voice assistant

<https://youtu.be/FrrxSg5ajgg> · <https://github.com/nightfury217836/ai-jarvis-iron-man-like-assistant>

Verdict: The least honest of the thirteen. A renamed copy-paste with a toy classifier and cloud STT dressed as "ML/NLP from scratch."

What it claims: Python + TensorFlow + NLP assistant built from scratch: custom intents.json, a trained ML model, speech recognition, pyttsx3 text-to-speech, real tasks (open apps, Google search, music, time/date, weather, notes, Wikipedia).

How it actually works (verified from source — the code was read): (1) microphone → `recognizer.recognize_google(...)` — **Google's cloud Web Speech API, not local ML**; (2) text is tokenized (Keras Tokenizer, vocab 1000) → a tiny Keras classifier (Embedding → pooling → 2 dense layers → softmax over intent tags), argmax with a 0.6 confidence floor, then a **random canned response** from intents.json; (3) an if/elif chain over 27 tags: `os.system("start chrome")`, search URLs, app launches, volume via pyautogui, shutdown/restart/lock, weather via **wtrr.in HTTP**, the Wikipedia package, notes appended to a text file, news = opens news.google.com; (4) pyttsx3 speaks (offline, Windows SAPI5); (5) an infinite loop with **no wake word**.

Stack: STT: Google Web Speech API (cloud — needs internet). ML: toy Keras embedding classifier trained 1000 epochs on a hand-written file; pickled tokenizer and label encoder. TTS: pyttsx3 (offline). Actions: os/webbrowser/subprocess, pyautogui, requests. Windows-only.

Evidence: The repo is real but dead — **0 stars, 0 forks, 8 commits** — and the code is internally branded "**JASMINE**" ("JASMINE AI ASSISTANT ONLINE"), i.e., copied and renamed from another project. The "ML/NLP" claim is technically true but it's a memorized lookup table; the 0.6 threshold is its only generalization guard. "Talks like a human" is false — every output is a canned random response. Reminders are text appended to a file; nothing ever fires.

Cost & limits: \$0. But: requires internet for STT, weather, Wikipedia, and searches — nothing meaningful works offline; Windows-only; always-recording loop with no wake word; anything outside the 27 tags fails; no LLM, no memory, no context at all.

Build 3 — manujademin: 100% offline JARVIS in Python

<https://www.youtube.com/watch?v=H0TbxYPXqzY>

Verdict: The most honest of the YouTube builds — a genuinely reproducible \$0 offline stack. But it's a voice chatbot, not an agent.

What it claims: A 100%-offline "self-made" JARVIS v1.0: openWakeWord wake word → faster-whisper STT → Ollama (Llama 3.1) brain → Piper TTS. Self-described as a "basic wrapper model" — open-source tools stitched together, nothing trained from scratch.

How it actually works: The canonical offline voice-assistant loop (components confirmed from the video description; no public repo found, so wiring is inferred from standard docs): mic stream → openWakeWord continuously detects the wake phrase on CPU in milliseconds → utterance captured → faster-whisper transcribes locally → POST to Ollama's local API (localhost:11434, Llama 3.1, likely 8B) → response text → Piper TTS (local ONNX neural voice) → speakers → loop returns to wake-word listening.

Stack: Wake: openWakeWord. STT: faster-whisper (Python). LLM: Ollama + Llama 3.1. TTS: Piper TTS. Memory: none disclosed (session context at best). Integrations: none — consistent with offline. All components are real, mature open-source tools.

Evidence: "100% offline" holds at runtime (only first-run model downloads need internet). The "wrapper model" framing is accurate and refreshing. Unverifiable: no repo, code, or independent reproduction found, so latency and demo robustness can't be confirmed. "Built from scratch" is marketing — it's integration work.

Cost & limits: \$0 software. Hardware is the real cost: Llama 3.1 8B needs ~5–6 GB RAM quantized; CPU-only inference means several-to-tens of seconds per response — laggy but usable; a modest GPU with 8GB+ VRAM makes it near real-time. Limits: the wake+Whisper+LLM+TTS chain stacks latency; Llama 3.1 8B is weak at tool/function calling — this talks, it doesn't act (no app control, no browsing); no persistent memory without adding a vector DB yourself; Piper voices sound robotic next to cloud TTS; STT errors cascade with no correction loop.

Build 4 — Software Gent: turn Hermes Agent into JARVIS (free)

<https://www.youtube.com/watch?v=RqwXffETwsc>

Verdict: The strongest *foundation* in the set — a real, free, open-source agent runtime. The video's "voice JARVIS" framing overstates what it gives you out of the box.

What it claims: Install the open-source Hermes Agent (Nous Research), pick a model, connect apps, build a UI, add voice and a "command matrix" — get a free, self-improving, persistent-memory JARVIS that can run offline on local models.

How it actually works: Install Hermes (Python runtime, **MIT license**) on your machine or a ~€5/mo VPS. It's a full agent runtime: CLI, messaging gateway (Telegram/Discord/WhatsApp and others), and an agent loop handling provider selection, tool execution (~47 built-in tools: shell, files, browser, web search, MCP servers, cron jobs, subagent delegation), context compression, and session persistence. Model is your choice: NVIDIA NIM free credits, OpenRouter, or local Ollama/LM Studio for offline. Voice input = **Wispr Flow**, a system-wide dictation hotkey (hold key, speak, cloud transcribes, text is typed into the CLI) — dictation, not conversational voice. Voice output = ElevenLabs TTS API reading the agent's text replies. The "command matrix" is saved prompt templates/skills/aliases, not a distinct technology. Memory is a real mechanism: a SQLite session database with FTS5 full-text search across all conversations, plus a "learning loop" where the agent writes Markdown SKILL.md procedure files that get injected into future prompts.

Stack: LLM: any (NVIDIA NIM / OpenRouter / Ollama / Anthropic / OpenAI). STT: Wispr Flow (cloud). TTS: ElevenLabs API. Voice-agent layer: none — chained dictation plus TTS. Automation: Hermes's own tools + cron + MCP. UI: Hermes CLI/web dashboard. Memory: SQLite + FTS5 + skill files. Hosting: self-hosted.

Evidence: Hermes is **VERIFIED** real — MIT, launched Feb 25 2026 as the successor to OpenClaw, ~185k–244k GitHub stars, NVIDIA selected it as the reference runtime for Nemotron 3 Ultra; multiple independent reviewers confirm it installs and runs. But it is not the polished product implied: a structural code review (Apr 2026) documents a 10,700-line god-object agent file, a "death spiral" bug where 212k+ token gateway sessions loop on API disconnects with no recovery, silent history deletion on failed summarization, and silent skill-loading breakage. Dominant pattern: **fail-silent on critical data operations** — the memory being sold can quietly lose data. "Gets smarter with use" is directionally true but oversold. The video description is also an affiliate funnel (Skool, Wispr Flow referral).

Cost & limits: A free path genuinely exists (Hermes free; OpenRouter :free models at 20 req/min, 50 req/day unfunded; Ollama local). But as shown it's not free: Wispr Flow Pro \$15/mo (free tier: 2,000 words/week), ElevenLabs metered (free ≈ 10 min/mo; Starter \$6/mo). It runs on your machine or VPS — laptop asleep means JARVIS is dead. Push-to-talk dictation only: no wake word, no barge-in, no phone calls — the Iron Man gap is large.

Build 5 — Nate Herkman: no-code JARVIS (Lovable + ElevenLabs + n8n)

<https://www.youtube.com/watch?v=KUvSzvFeZls>

Verdict: The most honest end-to-end product of the no-code builds — a real, documented integration. But it's a ~\$55–75/mo SaaS chain with no real memory and no local control.

What it claims: Zero-code JARVIS — Lovable builds the interface, ElevenLabs Conversational AI is the voice agent, n8n handles tool-calling workflows.

How it actually works: Lovable generates a React web app (Lovable Cloud hosted) with an embedded ElevenLabs voice widget. ElevenLabs runs the realtime voice stack (its STT → configured LLM → ElevenLabs TTS, with interruption handling) streamed to the browser. Tool calling: you define **custom tools** in the ElevenLabs agent (name, description, JSON schema) pointing at an **n8n Webhook node** (POST). Per call: user speaks → ElevenLabs LLM invokes the tool → POSTs parameters to n8n → the n8n workflow runs (AI agent node, APIs, 400+ integrations) → "Respond to Webhook" returns the result → the agent speaks it. n8n never touches audio; it's purely the tool backend.

Stack: LLM: the ElevenLabs agent's configured model (cost passed through on top of voice charges). STT/TTS: ElevenLabs. Voice layer: ElevenLabs Conversational AI widget/SDK. Automation: n8n webhook workflows. UI: Lovable React app. Memory: thin — in-session context plus an optional knowledge base; no real cross-session memory unless you build it. Hosting: Lovable Cloud + ElevenLabs cloud + n8n Cloud or self-hosted.

Evidence: Genuinely hangs together — n8n's own template library and independent course repos document exactly this architecture. "No code" is mostly honest (prompt-built UI, dashboard config, visual nodes), though webhook payload debugging is low-code in practice. Glossed gaps: **no persistent memory** (it forgets you between sessions), **no computer control** (only what n8n integrations expose), **no proactivity** (it only acts when spoken to), and it's a browser tab — close it and JARVIS is gone; mobile is a webpage, no native app, no wake word.

Cost & limits: ~\$55–75/mo floor for light use: Lovable Pro from \$25/mo (free: 5 build credits/day); ElevenLabs agents \$0.08/min + LLM passthrough (Starter \$6/mo → 75 min; Creator \$22 → 275 min; 30 min/day ≈ \$72/mo voice alone); n8n Cloud Starter €24/mo (2,500 executions — over the cap, workflows **halt until reset**, no overages). Self-hosted n8n Community is free and unlimited (~\$4–7/mo server) but then you own uptime, SSL, and updates. Each tool call adds 2–10 seconds of dead air. Three chained vendors = three independent failure points.

Build 6 — AIDVGENIUS: no-code JARVIS on Android mobile

<https://www.youtube.com/watch?v=CqoaGpZFxo8>

Verdict: A real, free, working voice-chat webpage — and zero percent JARVIS. A toy, honestly priced at \$0.

What it claims: On Android with no coding: copy a prompt from Telegram, paste into Google AI Studio to generate HTML, save as `index.html`, add a free OpenRouter key — a voice assistant on your phone that listens and answers.

How it actually works: A single static HTML page in the phone browser. Google AI Studio (Gemini) is just the **code generator** — you paste the creator's prompt, it outputs the page. (Why both AI Studio and OpenRouter is never explained; best-supported inference: Gemini writes the code, OpenRouter supplies runtime inference so no Google Cloud key or billing is needed.) The page: **Web Speech API** (`webkitSpeechRecognition` in Chrome Android — audio goes to Google's servers, needs internet) for mic input → `fetch()` POST to OpenRouter's chat-completions endpoint with your key and a JARVIS system prompt → `speechSynthesis` speaks the reply. Tap mic → wait → hear reply. No tools, no memory, no actions, no wake word.

Stack: LLM: OpenRouter free models via REST. STT: Web Speech API (cloud). TTS: `speechSynthesis` (device voices). Voice-agent layer: none (manual turn loop). Automation: none. UI: one HTML page. Memory: none. Hosting: none — a local file.

Evidence: The mechanism is real and reproducible — Web Speech plus fetch to OpenRouter is a known pattern; OpenRouter is CORS-accessible and its free tier needs no card (50 req/day unfunded). But it's a voice chatbot demo, not an assistant: zero agentic capability. "No coding" is brittle — generated Web Speech code commonly breaks (permission handling, voice-loading races, recognition cutting out on pauses), and debugging JS in a phone text editor is miserable. The actual code is gated behind Telegram so it couldn't be audited. **Security caveat:** the OpenRouter key sits in plaintext in the JS — acceptable for personal on-device use, but anyone copying the file gets the key; no backend proxy is mentioned.

Cost & limits: Genuinely ~\$0. Free models are \$0/token with limits of 20 req/min and 50/day unfunded (1,000/day after a \$10 lifetime purchase); free models rate-limit hard at peak and can rotate without warning. Recognition needs internet and Google's servers (not private, mediocre in noise); robotic Android voices; no background operation (the tab dies when the screen sleeps); recognition stops on pauses. Capability ceiling: it can *talk about* doing things; it can never do them.

Build 7 — @avoshuu: "Arcus," the local-first Chief Executive AI Officer

https://www.instagram.com/reel/DaZ_k0quhmK/ · follow-up
<https://www.instagram.com/reel/DdgKhHg0Yfb/>

Verdict: The boldest vision — one brain across every surface, fully local — with the thinnest verification. Consistent story, zero inspectable internals.

What it claims: A fully-local "Chief Executive AI Officer" built to reduce AuDHD cognitive load so one person can run a multi-branch company. Shown: a "Dynamic agent island" dashboard with **Research / Developer / Browser agents**; chat and PowerShell interfaces; surfaces spanning a desktop app, a mini-notch voice overlay, voice, smartwatch (ping or call it from the watch), Discord server control, and phone agents; it "proactively" calls him. Positioned as a "voice-based operating system" for computer, phone, and watch — managing apps, business, and finances.

How it actually works (reconstructed): Voice or text trigger on any surface → a local orchestrator ("proper reasoning harness," per his comments) → dispatches to the Research / Developer / Browser subagents on the dashboard → tool use: screen capture and vision ("she can legit see what's on my screen and use it as context"), hypervisor control ("spin up other VMs just by voice"), room and smart-home control, a booking system for his creative company, email drafting, project creation "on autopilot" → response via the voice overlay or a proactive outbound call.

Stack: Models: unspecified local models — the creator insists "fully local (not built with Claude or ChatGPT)" but names no model, no STT, no TTS, no vector or memory store. Agents: an unnamed "reasoning harness" plus Research/Developer/Browser subagents. Hardware: his own PC ("she has her own PC"); per comments, "now building servers." Surfaces: desktop app, voice mini-notch, smartwatch, Discord, phone. No repo, no open source ("this is not open"; only a limited web UI).

Evidence: Two reels two months apart show consistent, real-looking UIs (agent island, chat, PowerShell); his comment replies describe the same architecture repeatedly, and there's an active build-in-public arc on Twitch/Discord. Unverifiable: "fully local" can't be confirmed without code; skeptics allege it's a Claude wrapper ("you can literally do this in 2 days"); a reply claiming "we're now a full-backed YC startup" is unverified anywhere; a "13 years experience" bio claim was challenged and revised to 8. By his own admission the "gambling wheel" visuals are a "wheel of ideas" concept, not core function.

Cost & limits: Fully-local models capable of agentic multi-tool work need serious GPU hardware (\$1,500+ rig — "her own PC / building servers" is the tell). Screen-vision plus multi-agent loops burn context and latency budgets fast. Watch/phone gateways need push infrastructure and an always-on service. Proactive autonomous actions (VM spin-up, bookings) need safety guardrails or they become expensive mistakes. Closed source means no independent audit of the "local" claim.

Build 8 — @chloeshy.ai: the business JARVIS

<https://www.instagram.com/reel/DbBPiy4vcz4/>

Verdict: The clearest real business-operations demo in the set. Every named component independently exists; the quoted metrics are glossy and the glue is hidden.

What it claims: A voice "Jarvis" that runs her app business: a dashboard designed in Claude and generated in 5–10 minutes; an agent harness routed through hosted voice models; Kokoro TTS for speech; Playwright MCP so Claude sees her Chrome; RevenueCat plus Meta Ads MCPs; it reads business metrics aloud by voice and recommends operational actions (review backend PRs, research organic content). Comment "Jarvis" and she DMs a GitHub link.

How it actually works: (1) She vibe-codes a dashboard UI in Claude/Claude Code ("generated in 5–10 minutes"). (2) An agent harness connects Claude to hosted voice models for the speech loop. (3) Text-to-speech through Kokoro TTS. (4) Playwright MCP (Microsoft's official MCP server) gives Claude browser control and vision over her Chrome. (5) The RevenueCat MCP (official: mcp.revenuecat.ai, launched July 2025, 26 tools — **VERIFIED**) pulls app revenue and subscriber metrics; the Meta Ads MCP (several exist — ZuckerBot, Pipeboard/creatibi hosted, Ryze — all wrapping the Meta Marketing API, **VERIFIED**) pulls ad spend and creative performance. (6) Jarvis reports the 7-day figures by voice — 2,459 new downloads, \$4,289 revenue, \$475 ad spend, 1.5 ROAS — identifies street-interview creatives as top performers and slideshows as weakest, and recommends reviewing backend PRs and researching organic content angles.

Stack: Models: Claude (Anthropic; her #fable5 hashtag suggests the Fable 5 family). STT: hosted voice models (unspecified). TTS: Kokoro-82M — **VERIFIED** real: 82M-parameter open TTS (Apache 2.0), ~350MB, runs on CPU, free. Agents: Claude + Claude Code harness. Tools: Playwright MCP (browser), RevenueCat official MCP (app revenue), Meta Ads MCP (ads). Memory: none named. Hardware: desktop, multi-monitor. Surfaces: dashboard + voice. The GitHub link she distributes is the **Stanford + Lambda Labs OpenJarvis** project ("OpenJarvis: Personal AI, On Personal Devices," **VERIFIED** via arXiv) — her "Jarvis" is substantially a wiring of OpenJarvis plus commodity MCP servers: a legitimate build, not from-scratch invention.

Evidence: Strong on components — every named piece independently exists, and the metric-reporting demo is highly plausible (RevenueCat's Charts API and the Meta Marketing API expose exactly those numbers). Weak on craft: comments were a fulfillment funnel (dozens of "Jarvis" → identical DM replies); substantive technical debate was thin; one skeptic noted Claude Code's screen-capture reading is flaky (true — browser-vision degrades agent reliability). Math caveat: $\$4,289 \div \$475 \approx 9x$, not the 1.5 ROAS she states — treat quoted figures as illustration, not audited. "5–10 minutes" hides the real glue work (API keys, MCP config, auth).

Cost & limits: Hosted Claude plus voice-model API usage is the ongoing cost. Playwright browser automation is flaky and slow. Meta Ads MCPs with write tools can spend real ad dollars — the confirmation model matters enormously. The Meta Marketing API needs an approved app and token management (OAuth refresh pain). RevenueCat only pays off if you actually have an app with purchases. Replicating her exact setup means reproducing her undisclosed harness glue — the shared repo is OpenJarvis, not her private config.

Build 9 — @alexcandothat: the DIY pipeline ("intelligence = context + relevancy")

<https://www.instagram.com/reel/Dc2C4fHnr4Y/> · follow-up
<https://www.instagram.com/reel/DdZ7RM9JYXd/>

Verdict: The most transparent build in the set — a 28+-part build-in-public series with on-camera hardware demos, including the failures. The memory thesis is the single best idea across all thirteen.

What it claims: A from-scratch home-built JARVIS with a fully documented pipeline: Plaud recorder → LocalWhisper transcription → Obsidian knowledge graph → Hermes agents on a Mac Mini → "Fletcher" retrieval → Raspberry Pi speech loop → Amazon Echo voice. Core thesis: "intelligence = context + relevancy." Follow-up: replacing the Pi's microSD with a USB SSD to run more offline LLMs.

How it actually works: (1) She records everything she says with a Plaud audio recorder. (2) Audio goes to LocalWhisper (**VERIFIED:** real local Whisper transcription apps for Mac, 100% offline). (3) Transcripts feed a large Obsidian graph — markdown notes linked in Obsidian's graph view — that "sorts information for her agents." (4) "Hermes agents" run on a Mac Mini — this maps cleanly to Nous Research's **Hermes Agent** (**VERIFIED:** MIT open-source agent framework, Feb 2026, model-agnostic across 30+ providers including Ollama/LM Studio, built-in skill learning and messaging gateways). Her phrase "keeps the loop updated regardless of the AI model used" matches Hermes Agent's documented model-agnostic design. (5) "**Fletcher**" is her own name for the retrieval system on the Mac Mini — also her build-updates site and Pi dashboard; it is NOT a public component (no repo found — treat Fletcher internals as unverified). (6) A Raspberry Pi 5 handles the interactive speech loop; earlier parts show an M5Stack Atom Echo (**VERIFIED** \$13 ESP32 voice dev kit) as the listen/speak device, a ceiling-mounted projector plus overhead camera doing hand-tracking for a projected desk dashboard (two "brains" — one Pi for hand-tracking, one for requests — after frame-rate drops when speaking), plus an Amazon Echo for vocal processing. (7) Later parts: a Flipper Zero (**VERIFIED**) captures the projector remote's IR codes and the Pi replays them — "Fletcher, turn on the projector" demonstrably works on camera; microSD → USB SSD swap (the AI HAT blocked the SD slot) for more LLM headroom; an ArduCam narrow-depth-of-field camera upgrade for better hand tracking.

Stack: Capture: Plaud recorder (**VERIFIED:** Plaud NotePin, \$159, 300 free AI transcription min/mo). STT: LocalWhisper (offline). Memory: Obsidian vault + graph. Agents: Hermes Agent on Mac Mini. Retrieval: "Fletcher" (her own integration, internals unverified). Voice: Amazon Echo + M5Stack Atom Echo. Hardware: Mac Mini, Raspberry Pi 5 + AI HAT + M.2 HAT+ + USB SSD, overhead camera, ceiling projector, Flipper Zero, ArduCam. Surfaces: voice (Echo), projected hand-tracking dashboard, Fletcher website.

Evidence: Strongest of the reels: 28+ parts over ~6 weeks with on-camera demos — live hand-tracking dashboard, Flipper Zero IR capture visibly powering on the projector, the SSD clone shown at 61%, supporter-bought parts via wishlist. She shows failures honestly (frame-rate drops while speaking, "really shitty" camera rig, "I have no clue what I'm doing" posts) — a credibility plus. Skepticism: "your first brain doesn't actually work," "vibe coded"; one technical commenter suggested converting the Obsidian graph to a vector DB (a valid scale critique of file-based retrieval). Gloss: the "intelligence = context + relevancy" framing is philosophy, not architecture; and despite "offline" language, Plaud transcription and the Amazon Echo both involve cloud services — the Echo especially puts voice through Amazon. Fletcher's retrieval mechanics are a black box.

Cost & limits: Hardware totals several hundred dollars: Mac Mini (~\$599+), Pi 5 + HATs + SSD + camera + projector + Flipper Zero (~\$150) + Plaud (\$159) — she funds parts via supporter wishlist. The Pi is the bottleneck: hand-tracking frame rate visibly drops while the AI speaks; offline LLMs on a Pi are weak, so real inference lives on the Mac Mini. Obsidian-as-graph is file-based retrieval — fine at personal scale, degrades as notes grow. Always-on Plaud recording brings battery, privacy, and consent overhead (it records other people too). The Echo dependency contradicts the offline narrative. Replicating her setup means re-deriving her undocumented Fletcher glue.

Build 10 — @lukebuildsai: the 6am call (Azaris)

<https://www.instagram.com/reel/DdRp2rqpwHI/>

Verdict: A scripted advertisement — but the product it points to is real. Evaluate the product, not the skit.

What it claims: A 6am phone call from "Jarvis" reporting overnight work: handled emails, replied to DMs, did support, got signups — "done before I was even up." Comment "Jarvis" and he'll send "the platform where you can build your own."

What it actually is: A scripted dramatization with a lead-gen CTA. The destination is real: Luke Cutting's bio reads "Cofounder @azaris.ai | Software Engineer," and the DM'd platform is **Azaris (azaris.ai)** — live site, **pricing \$97/month DIY** (bring your own model key, no token markup; cloud browser + persistent memory included; channels: Telegram, Slack, Discord, Teams, voice; **"approval cards"** before big decisions). 1,000+ integration catalog plus native connectors plus a cloud-browser fallback. A referral loop (\$10/mo off per referral, up to 5) explains the aggressive funnel. Suggestive but unconfirmed: the GitHub org **azaris-ai-dev/openclaw** implies it's built on **OpenClaw**, the open-source personal-assistant agent framework. **Not verifiable:** that the 6am call happened through Azaris — no logs, no screen recording, no architecture shown. It's a skit, not a demo. (Irony noted: his own Aug 31 post reposting another creator's UI says "I just couldn't tell from the video whether anything was actually connected behind it.")

Stack (verifiable): Azaris: hosted agent platform, BYOK models, cloud browser, persistent memory (typically vector-store RAG — degrades over time, verify before trusting), multi-channel messaging, approval cards.

Evidence: Plausible product, marketing-grade demo. Comments: ~99% keyword spam plus awe, zero answered technical questions, no independent reviews found on the web. One step above vaporware because the destination product exists.

Cost & limits: \$97/mo plus your own token costs — an "always-on" agent with a cloud browser burns tokens fast. Cloud-browser agents are flaky on login-walled sites (2FA, CAPTCHAs). "Persistent memory" claims need verification.

Build 11 — @blakeestephens: the driving call

<https://www.instagram.com/reel/DdjH8Q0yzmX/>

Verdict: Pure marketing. A course funnel with no verifiable system behind it.

What it claims: While driving, his JARVIS calls to report: signed clients, resolved a frozen bank account, sent documents, booked another lead. Caption: "JARVIS built my whole business. Comment 'JARVIS' if you want your own AI to do the same." Bio: "I built a \$30K/mo AI agency w/ JARVIS ... Have JARVIS build your AI Agency (free course)."

What it actually is: A course/lead funnel, not a system demo. His entire recent feed (15+ reels) is the same template with the same CTA. Comments are keyword spam plus auto-replies ("Nice! Check your DMs!"); the one substantive question ("Is he a chatbot or is he from your computer?") got no technical answer. One post ("Jarvis really fucked me here...") uses a fake "mistake" as bait for the same CTA. **Zero architecture ever shown** — no screens, no stack, no workflow. The "\$30K/mo AI agency" is unverifiable; the deliverable is a "free course," the standard upsell vehicle.

Stack: None verifiable. No model, framework, tools, or integrations named anywhere.

Cost & limits / risks: The "free course" is the top of a funnel — expect an upsell (typically \$500–\$5,000 in this niche) and/or an affiliate stack. It conflates lead-gen automation (real, modest) with autonomous deal-closing (not real). Risk: time and money sunk into prompt-level automation dressed as autonomous agency.

Build 12 — @realrobertgutierrez: "Ghost X1" (abliterated model)

<https://www.instagram.com/reel/Dd0-lomR8mR/>

Verdict: The technique is real; the product is vaporware; the premise is wrong for a personal JARVIS. Skip it.

What it claims: "Today is the day that will be written in history books... We just made it possible for you to access an abliterated AI model directly from your Claude Code or Codex terminal." Comment "GHOST" to try it free.

What it actually is: (1) **"Abliterated" is a real technique (VERIFIED):** weight-modification that surgically removes the refusal direction from an open-weight LLM — persistent across prompts, not a jailbreak. Established in the open-source community (huihui-ai's abliterated Qwen models; the startup Abliteration.ai made news Aug 31, 2026). (2) "Inside Claude Code/Codex terminal" is technically plausible — these tools can be pointed at an OpenAI-compatible endpoint serving any model. (3) But **"Ghost X1" has no verifiable existence:** zero web hits — no HuggingFace repo, no weights, no docs, no GitHub, no website. Follow-up posts never name a base model or parameter count; they repeat "first 1,000 who comment GHOST." (4) The feed is affiliate/keyword marketing (#VercelPartner, #higgsfieldpartner tags; every post a comment-keyword funnel). The only technical question in the thread — "Which ablated model?" — went unanswered; the other ~60 visible comments are the word "Ghost." Honest read: vaporware or a thin wrapper around someone else's abliterated open-weight model, gated behind the funnel.

Stack: None verifiable for Ghost X1. (The real-world recipe for what's described: an abliterated open-weight model — e.g., a Huihui-Qwen abliterated variant on HuggingFace — served via vLLM/Ollama behind an OpenAI-compatible endpoint.)

Why not to build on it: Abliterated models are fine-tunes of open models already weaker than frontier models at instruction-following, tool use, and long-horizon agency — the exact capabilities a JARVIS needs. Removing refusals doesn't add competence. An always-on agent with your email, money, and accounts that *cannot refuse* is a liability — prompt injection and accidental harmful actions get worse, not better. Plus unvetted third-party infrastructure in the loop.

Build 13 — @jonathonmj: "Z.E.R.O"

<https://www.instagram.com/reel/DZcSLefuxRU/>

Verdict: The best *framing* in the set; no verifiable system. Steal the philosophy, ignore the product.

What it claims: A theatrical "brain" UI. Z.E.R.O: 32 tools, persistent memory, operates CRM, money, inbox, and web, multi-agent workflows. Key line: "an LLM or coding agent is a tool; JARVIS should be an operator with purpose, tools, memory, permissions, and feedback loops."

What it actually is: (1) The framing is genuinely good — the tool-vs-operator distinction is the correct mental model and matches serious agent architecture. (2) But no verifiable system exists behind the theatrics: web search returns nothing relevant. Bio: "AI Consulting | Agency Owner | The Original Z.E.R.O | Book your strategy call here" — monetization is consulting calls. (3) His own feed undercuts the "real system" reading: a June 6 post asks "Should we open source his repo?" (no repo ever appeared); "Guide dropping in less than 2 weeks" → "Guide dropping soon..." — perpetual soon-teases. Tagged #claudecode throughout — if anything real exists it's probably a Claude Code + MCP setup, but no architecture, tool list, or memory design has ever been published. (4) Comments: awe, "link plz," and skeptics — "this looks sk," "is there any repo?", "what is this visualization?" — plus one pointed critique: "shoutout to all these guys showing this type of shit and not showing what their agents are actually generating in terms of business outcomes." No technical answers from the creator.

Stack: None verifiable. The #claudecode + 32-tools hints suggest Claude Code + MCP servers, but that's inference, not evidence.

Cost & limits / risks: The risk is **cargo-culting** — copying the aesthetic (brain UI, "32 tools") without the engineering (idempotent tools, approval gates, memory hygiene, eval loops). A visualization proves nothing about reliability. A "book a strategy call" CTA on an unverified system is a red flag for paid advice of unknown quality.

3. The combined architecture: best of all 13

Every serious build converges on seven layers. Here is the reference architecture assembled from the best idea at each layer, with attribution.

#	Layer	Best idea	From	How it works
1	Reasoning brain	Model-agnostic frontier model with hot-swap; BYOK	Zubair (hot-swap), Hermes (model-agnostic), Azaris (BYOK)	The brain is a config choice, not a moat. Use the best hosted model you can afford; keep the provider swappable (endpoint + model string). Local Ollama fallback for offline/private tasks (manujademin).
2	Memory	Three-tier: capture pipeline + session store + learned skills	Alex (capture → transcribe → graph), Hermes (SQLite+FTS5, SKILL.md learning loop)	(a) Capture everything: voice notes → local Whisper → markdown vault/Obsidian graph. (b) Session store with full-text search across all conversations. (c) The agent writes reusable procedure files ("skills") from experience. Intelligence = context + relevance.
3	Tools / hands	MCP servers + built-in agent tools + no-code integrations	Chloe (RevenueCat/Meta Ads/Playwright MCPs), Hermes (47 built-in tools), Nate (n8n 400+ integrations)	Business-data MCPs for metrics; built-in tools for shell/files/browser; n8n webhooks where a visual integration beats hand-written code.
4	Agent loop	Cron + overnight batch + proactive briefs + approval cards	Luke/Azaris (overnight work, approval cards), Hermes (cron), Zubair (morning brief)	Scheduled runs do the work while you sleep; a morning brief reports it; approval cards gate anything consequential (money, messages, ad spend).
5		Wake word → STT → TTS; realtime layer for conversation; telephony only if earned	manujademin (openWakeWord/faster-whisper/Piper), Nate (ElevenLabs Conversational AI), Zubair (Retell), Alex (Echo)	Local pipeline for private/cheap; ElevenLabs Conversational AI for natural realtime voice; Retell-style phone calls only when outbound calling proves its value.
6	Surfaces	One brain, many surfaces	Arcus (desktop/watch/Discord/phone), Hermes (messaging gateways), Chloe/Zubair (dashboard)	WhatsApp/Telegram first, then desktop overlay, web dashboard, watch pings. The brain is singular; surfaces are interchangeable.
7	Privacy tiers	Local-first for sensitive, hosted for capable	manujademin (offline), Alex (local Whisper/graph), Arcus (local-first claim)	Voice and notes transcribed locally; notes stored locally; frontier hosted models only for reasoning over retrieved context.

The operator contract (from Z.E.R.O — the framing worth stealing): an LLM is a tool. JARVIS is an **operator**: it has a *purpose* (your goals), *tools* (layer 3), *memory* (layer 2), *permissions* (approval cards, layer 4), and *feedback loops* (verify → correct, layer 4). Any build missing one of the five is a demo, not an operator.

4. Substance-vs-marketing scorecard

Build	Substance	Basis
9. Alex — DIY pipeline	●●●●●	28+ part build-in-public, on-camera hardware demos, honest failures
5. Nate — no-code voice stack	●●●●○	Real documented integration; pricey, no memory
8. Chloe — business JARVIS	●●●●○	Every component verified real; metrics glossy, glue hidden
4. Hermes Agent tutorial	●●●●○	Runtime is real and free; documented fail-silent bugs; voice is bolted on
1. Zubair — Fable 5.1	●●●●○	Real components; internals gated behind paywall
3. manujademin — offline	●●●●○	Honest, reproducible; a chatbot, not an agent
7. Arcus	●●●●○	Consistent 2-month vision; zero inspectable internals
13. Z.E.R.O	●●○○○	Best framing; no verifiable system

Build	Substance	Basis
6. AIDVGENIUS — mobile	●●○○○	Working toy; zero agentic capability
10. Luke / Azaris	●●○○○	Real \$97/mo product; the reel is a scripted ad
2. SouvikChai	●○○○○	Copied code, toy classifier, misleading claims
11. Blake — driving call	●○○○○	Pure course funnel
12. Ghost X1	●○○○○	Vaporware; premise is wrong for this goal

5. What NOT to copy

1. **Course and lead funnels** (Blake; Z.E.R.O's consulting; Ghost X1's keyword gates). No system has ever been shown. Money flows toward the guru, never toward your build.
2. **The Ghost X1 premise.** Uncensored models are weaker at instruction-following, tool use, and long-horizon agency — the exact capabilities JARVIS needs — and removing the refusal boundary on an always-on agent with access to your email, money, and accounts is a liability, not a feature.
3. **Toy "AI"** (SouvikChai). A 27-class intent classifier with canned responses is not an assistant and teaches nothing about the real architecture.
4. **Cargo-cult aesthetics** (Z.E.R.O's brain UI, "32 tools"). A visualization proves nothing about reliability. Reliability comes from idempotent tools, approval gates, memory hygiene, and eval loops — the boring parts no reel shows.
5. **Memory-less SaaS chains** (Nate's stack as a brain). ElevenLabs + n8n is a fine *voice front-end*; without persistent memory it forgets you every session. Don't mistake the mouth for the mind.
6. **"Fully local" as a purity test** (Arcus's unverified claim; Alex's Echo-shaped hole). Local-first is a privacy tier, not a religion — the winning pattern in every real build is local capture plus hosted reasoning.
7. **Scripted ads as evidence** (Luke's 6am call). Overnight batch processing plus a morning brief is a real, stealable pattern; the skit proves nothing about the product.

6. Build plan — phased for a non-programmer with AI coding tools

Design constraints: you build with AI coding tools, not hand-written code. WhatsApp and Android first (already connected). Local-first where it matters. Tight budget — every dollar weighed against debt recovery. And the job is memory and proactivity, not voice realism.

Phase 1 — Weekend V1: "The Remembering Assistant" (~\$5-10/mo)

Goal: a system that **remembers you across weeks** and **briefs you without being asked**. Nothing else matters yet.

- **Always-on runtime:** Hermes Agent (free, MIT) on a ~\$5/mo VPS — or a machine that genuinely stays awake. A laptop that sleeps is not a server.
- **Interface:** WhatsApp via Hermes's messaging gateway. One brain, first surface.
- **Memory:** a markdown vault (Obsidian-compatible) as the long-term store; Hermes's session memory for conversation recall. Start the highest-leverage habit in the whole plan: **a 2-minute end-of-day voice note**, transcribed locally with Whisper, appended to the vault. This is Alex's pipeline at 1% of her hardware.
- **Tools (two only):** Google Calendar + Gmail read (both already connected), plus web search. Two tools that work beat ten that don't.
- **Scheduler:** a morning brief to WhatsApp every day — calendar, open commitments, money-relevant items, timed around (never over) prayer times. A short nightly shutdown note: what got done, what's open.
- **Model:** start cheap via OpenRouter (free tier: 50 req/day unfunded); upgrade to a frontier model when token burn justifies it. The brain is swappable — act like it.
- **Done looks like:** it references something you told it three weeks ago without prompting; the brief arrives daily without fail for 30 days.

Phase 2 — Voice + money tools (~\$25–50/mo, only when Phase 1 is boring)

- **Voice loop:** ElevenLabs Conversational AI as the realtime voice front-end (~\$0.08/min — budget it against actual use), or the local Whisper + Piper/Kokoro pipeline to stay near \$0. Outbound phone calls (Retell-style, ~\$0.11–0.16/min) only if a concrete use case earns it.
- **Money tools:** MCP-style integrations for what earns: Amazon Seller Central data, ad metrics — Chloe's loop, pointed at your business. **Approval cards before anything spends money or messages anyone.**
- **Memory upgrade:** the agent starts writing its own SKILL.md procedure files (Hermes learning loop) — it gets smarter from use instead of re-learning.

Phase 3 — The operator (one-time hardware ~\$100–200 + tokens)

- **Home server** (mini-PC / Mac-Mini class): Hermes running 24/7, local Ollama fallback for offline and fully private tasks.
- **More surfaces:** desktop overlay, watch pings, web dashboard — Arcus's "one brain, many surfaces," earned in order.
- **Subagents:** Research / Developer / Browser agents with feedback loops — only after the single agent is reliable. Multi-agent multiplies capability *and* failure modes.

V1 vs. later — stated plainly

V1 is **memory + briefs + two tools on WhatsApp**. Voice calls, screen control, business automation, multi-agent, and local LLM hardware are all later. **Do not buy hardware in Phase 1**. The most expensive mistake in all thirteen builds is buying the costume (dashboards, GPUs, gadgets) before the memory works.

Cost cheat sheet (verified ranges, Sept 2026)

Piece	Cost
Hermes Agent runtime	\$0 (MIT) + ~\$5/mo VPS
Cheap models (OpenRouter free tier)	\$0 (50 req/day)
Frontier model (Claude Fable 5.1 API)	\$10/M input, \$50/M output tokens
ElevenLabs voice agents	\$0.08/min + model passthrough (~\$72/mo at 30 min/day)
n8n (self-hosted)	\$0 + ~\$5/mo server; Cloud €24/mo (2,500 executions, halts at cap)
Lovable UI	\$25/mo Pro
Retell phone calls	~\$0.11–0.16/min + \$2/mo number
Azaris (managed alternative)	\$97/mo + your tokens

Piece	Cost
Offline stack (Ollama/Whisper/Piper)	\$0 software; capable GPU hardware if you want speed
Kokoro TTS	\$0 (open, runs on CPU)

7. Open questions to verify before building

1. **Hermes's fail-silent memory bugs** (silent history loss, skill-loading breakage) — check the current release's issue tracker before trusting it with anything money-relevant. Memory that silently deletes itself is worse than no memory.
2. **WhatsApp gateway path** — Hermes's messaging gateway vs. official API terms; confirm the compliant route for your account.
3. **Azaris vs. self-hosted** — if Phase 3 ever wants managed hosting, price Azaris's \$97/mo against ~\$10/mo self-hosted plus your own ops time.
4. **Fletcher-shaped gap** — Alex's retrieval layer is her own undocumented glue; when your vault grows past a few hundred notes, plan the Obsidian-graph → vector-database migration the commenters flagged.

Sources: the 13 videos/reels listed above; hermes-agent.nousresearch.com; Nous Research docs; OpenJarvis (Stanford + Lambda Labs, arXiv); Kokoro-82M (hexgrad, Apache 2.0); Microsoft Playwright MCP; RevenueCat MCP (mcp.revenuecat.ai); Meta Ads MCP servers (ZuckerBot, Pipeboard/creatibi, Ryze); ElevenLabs, n8n, Lovable, Retell, and Wispr Flow pricing pages; Anthropic Fable 5.1 release coverage; Abliteration.ai / abliterated-model community docs.